



Open ACE



Self-Hosted AI Coding Agent Workspace & Governance Control Plane

One place for every AI coding tool your team uses — keys, cost, and risk under control.

Apache 2.0

v1.2.0

Self-Hosted First

THE PROBLEM

When AI Coding Tools Meet Teams, the Real Work Begins



Keys Everywhere

API keys scattered across laptops and remote machines — high leak risk, hard to rotate, no traceability



Hard to Run Remotely

AI CLIs belong on intranet dev, staging, or GPU machines; repeatedly sharing SSH credentials is clumsy



Fragmented Tools

Claude Code, Qwen Code, Codex, OpenClaw each on their own — no unified entry point or history



Cost Blind Spot

How many tokens, how much money, who is using what, any overspend — largely a black box



No Audit Trail

Compliance, audit, quotas, and sensitive-content screening are missing — production is out of reach



Unproven ROI

AI payback is unclear, efficiency gains are unquantified, and decisions lack evidence

One Platform, Double Value

Open ACE = AI Coding Workspace + Governance Control Plane

Developers use Claude Code, Qwen Code, Codex, OpenClaw and more from one browser workspace, running them on the team's own remote machines;

administrators centrally manage API keys, access, cost, quotas, audit, and compliance — fully self-hosted, data never leaves your network.



Unified Entry

Many AI CLIs, one browser workspace, one history, one governance layer



Remote Execution

Remote Agent runs CLIs on target machines — no credential handoffs



Key Security

Keys stay encrypted server-side; sessions get short-lived proxy tokens



Governance Visibility

Tokens, cost, quotas, audit, compliance, ROI — at a glance

TWO MODES

Two Modes, Two Audiences



Work Mode — Your AI Super Assistant

For every developer: a smooth AI experience

- Multi-tool: Claude Code / Qwen Code / Codex / OpenClaw / ZCode in one place
- Remote workspace: pick a machine in the browser, start AI sessions, terminal, file browsing
- Smart sessions: history, recovery, context memory, cross-tool sync
- Prompt library: share best prompts across the team, reuse in one click
- Quick search: find any past conversation, knowledge never lost



Manage Mode — Data-Driven AI Governance

For administrators: full insight and control

- Usage visibility: token trends, cost analysis, usage heatmaps
- API key governance: encrypted storage, proxy-token calls, no key distribution
- Smart alerts: quota warnings, anomaly detection, overspend notices
- Compliance audit: sensitive-content detection, traceable history, CSV reports
- Multi-tenant + ROI: department isolation and quantified payback

Eight Core Capabilities, Full AI Coding Lifecycle



Agent Control Plane

Manage tools, users, machines, keys, quotas, cost and audit in one place — without replacing any CLI



Multi-CLI Adapters

Native adapters for Claude Code, Qwen Code, Codex, OpenClaw, ZCode — with session recovery and permission modes



Remote Agent

AI CLIs run on Linux/macOS/Windows machines with in-browser terminal and VSCode proxy



API Key Proxy

Real keys encrypted server-side; remote agents hold short-lived tokens with unified quotas



Autonomous Dev

End-to-end workflow: plan → code → PR review → report; retry/fork/milestones, gVisor/Kata sandbox



Model Gateway

LiteLLM-compatible gateway, vendor-neutral, keeps quotas and attribution intact



Cloud-Native

One-command Kustomize deploy, 3 replicas + HPA + Prometheus metrics, OpenShift-compatible



Enterprise Integration

SSO / Feishu / DingTalk, multi-tenant permissions, Kubernetes, reverse proxy

HOW IT WORKS

Remote Agent + API Key Proxy: Secure and Efficient

Browser (React SPA)

Work Mode (everyone) · Manage Mode (admins)



Flask API Server (self-hosted)

23 blueprints · encrypted key storage · quotas/audit/compliance · issues short-lived proxy tokens



Remote Agent (daemon on your machines)

HTTP polling · CLI subprocesses · WebSocket PTY terminal · session sync
(Claude/Qwen/Codex/OpenClaw)

Why this design?

- Real API keys never leave the server
- Remote machines hold only short-lived tokens
- Tokens are quota-able, revocable, traceable
- Works on intranet / private-network machines
- Terminal screens recover after disconnects
- Vendor-neutral, LiteLLM-ready

✦ Built for teams that need self-hosting, intranet machines, centralized key proxying, team quotas, and traceable audit

Kubernetes / OpenShift Cloud-Native Integration

HA Deployment on K8s

- One-command Kustomize deploy (kubectl apply -k k8s/)
- 3 replicas + HPA autoscaling (3→10 pods)
- PostgreSQL / Redis StatefulSets, persistent
- /livez /readyz probes + Prometheus /metrics
- PDB + anti-affinity + graceful rolling updates
- Least-privilege NetworkPolicy + non-root



AI Sandbox Isolation

- Backends per tenant/project: POSIX / remote / sandbox
- gVisor / Kata dual-tier secure runtimes
- Non-root + read-only rootfs + seccomp
- CPU / memory / PIDs / storage limits + TTL
- Default-deny egress + FQDN allowlist
- Fail-closed: refuse and audit on weak isolation



Multi-User Isolation

- Per-user WebUI in its own sandbox pod, token identity
- No host OS accounts / sudo chain / host credentials
- Tenant-scoped OS isolation, revocable
- Stable UUIDs across rebuilds; 60s quota sweeps
- Session snapshots survive pod recycling

Value for Developers, Managers, and the Enterprise



For Developers

- Every AI coding tool in one entry
- Remote machines ready to use, no setup
- Session history preserved, knowledge retained
- Focus on code — keys and envs handled by the platform



For Managers

- Tokens / cost / quotas on one screen
- Alerts and anomaly detection, early warnings
- Audit and report exports, production-ready
- Quantified ROI — AI payback made clear



For the Enterprise

- Data and keys never leave your network
- Self-hosted deployment, compliance-ready
- Multi-tenant isolation, per-department accounting
- Open source control, no vendor lock-in

Positioning vs. Market Alternatives

Capability	Open ACE	LLM Gateways (LiteLLM etc.)	AI IDEs/Plugins (Cursor etc.)	DIY Scripts
Role	Workspace + control plane	Routing & billing	Solo coding UX	Ad hoc
Self-hosting	✔ Default self-hosted	✔	✗ Mostly cloud	✔
Multi-CLI unified entry	✔ Claude/Qwen/Codex etc.	✗ Model layer only	⚠ Walled gardens	✗
Remote Agent execution	✔ In-browser remote machines	✗	✗	⚠ DIY
Keys never leave server	✔ Short-lived proxy tokens	⚠ Via gateway	✗ Local keys	✗
Cost/quota/audit/compliance	✔ Unified governance	⚠ Billing only	✗	✗
Multi-tenant / SSO / Feishu	✔	⚠ Partial	✗	✗
K8s-native & AI sandboxing	✔ HPA + gVisor/Kata	⚠ Container only	✗	✗ DIY
License	Apache 2.0	Varies	Closed	—

KEY DIFFERENTIATORS

Why Open ACE?



Self-Hosted by Default

Keys and session data stay in your own network and database — private, compliant, no cloud dependency.



Control Plane, Not a CLI

Replaces none of your tools: brings the CLIs your team already uses under unified access, cost, and audit.



Short-Lived Proxy Tokens

Real LLM keys stay encrypted server-side; sessions hold expiring tokens — quota-able, revocable, traceable.



Real Remote Execution

AI CLIs run on dev/staging/GPU machines; in-browser terminal + VSCode proxy, no repeated SSH handoffs.



Cloud-Native, Sandboxed

HA Kubernetes deploy; autonomous dev and multi-user workspaces run in gVisor/Kata sandboxes, fail-closed.



Vendor-Neutral + Tenancy

LiteLLM-compatible gateway, unified multi-CLI governance, tenant isolation and accounting.

Who Needs Open ACE Most?

Engineering Teams

Already on Claude Code / Qwen Code / Codex and want one entry, shared prompts, and session continuity.

AI Platform Teams

Building internal AI tooling that needs key proxying, remote execution, and one governance plane.

DevOps / IT Teams

Own self-hosted infrastructure, audit, and access control; need private deployment and multi-tenancy.

Security & Compliance

No shared public demos, no scattered keys — needs audit trails and exportable compliance reports.

GPU / Compute Teams

AI CLIs must run on GPU or staging machines; browser remote workspaces and terminals.

Multi-Department Enterprises

Need department isolation, independent quota accounting, SSO, and Feishu/DingTalk integration.

Start in 5 Minutes, Roll Out Incrementally

One-Command Deploy (Recommended)

```
git clone https://github.com/open-ace/open-ace.git
cd open-ace
docker compose up -d --build

# open http://localhost:19888
```

Other deployment options

- Source install (local development)
- Kubernetes Kustomize (3 replicas + HPA + Prometheus)
- Linux systemd / macOS launchd remote agents
- SQLite (single box) or PostgreSQL (production)

Incremental Rollout

- Step 1** Try locally with Docker; evaluate both modes
- Step 2** Onboard team keys; enable the API Key proxy
- Step 3** Deploy Remote Agents to dev/staging/GPU hosts
- Step 4** Move to HA Kubernetes; enable AI sandboxing
- Step 5** Quotas, audit, SSO + compliance & ROI reports

Bring AI coding agents in.

Keep keys, cost, and risk under control.

Open ACE — the self-hosted workspace and control plane for AI coding agents.



Website

www.open-ace.com



Repository

github.com/open-ace/open-ace



Docs

open-ace.github.io/open-ace-docs



Feedback

[GitHub Discussions](#)